

BIGENIA: Implementación de una Herramienta de Inteligencia de Negocios basada en Lenguaje Natural para PyMEs

Diseño de una arquitectura de generación dinámica de dashboards para democratizar el acceso a datos.

Lucas J. Martinez, Nahuel A. Carro
Prof. Javier D. Rabuch

11 de julio de 2026

Resumen

Este trabajo presenta BIGENIA, una arquitectura de Inteligencia de Negocios Generativa (GenBI) diseñada para democratizar el análisis de datos en Pequeñas y Medianas Empresas (PyMEs) eliminando la barrera de los intermediarios técnicos.

Se implementa un sistema multiagente capaz de transformar consultas en lenguaje natural en sentencias SQL y generar visualizaciones interactivas mediante la librería Plotly. La validación de la arquitectura se realiza sobre una base de datos relacional, aplicando una metodología de evaluación *LLM-as-a-Judge* sobre un conjunto de prueba de 50 consultas categorizadas por dificultad, incluyendo casos fuera de dominio.

Los resultados empíricos demuestran un cumplimiento robusto de los objetivos funcionales: el sistema exhibe un desempeño sobresaliente en la resolución de consultas de complejidad baja y media, y demuestra una alta resiliencia al mitigar consistentemente peticiones ajenas al contexto analítico.

Si bien se identifica una degradación en la precisión ante consultas que requieren múltiples uniones (*JOINs*), este comportamiento establece con claridad el límite operativo de la versión actual sin afectar su utilidad estratégica. Se concluye que BIGENIA corrobora la viabilidad de las interfaces basadas en lenguaje natural para dotar de autonomía analítica real a los usuarios de negocio.

Palabras clave: Inteligencia Artificial Generativa, Business Intelligence, Lenguaje Natural, PyME, Agentes de IA, Visualización de Datos.

1. Introducción

Las pequeñas y medianas empresas (PyMEs) representan un pilar fundamental de las economías latino-americanas; sin embargo, su acceso a herramientas de

inteligencia de negocios y analítica de datos sigue siendo limitado. Según Tovar [1], los principales obstáculos para la adopción de soluciones de *Business Intelligence* (BI) en este segmento empresarial son el costo elevado de las plataformas, la falta de personal cualificado y la complejidad de las integraciones con sistemas existentes.

Esta brecha tecnológica se profundiza cuando se considera que, a diferencia de las grandes corporaciones, las PyMEs carecen de departamentos de TI dedicados capaces de gestionar infraestructuras de datos complejas [2]. A esto se suma la incompatibilidad entre sistemas heredados y las nuevas tendencias tecnológicas, lo cual constituye una barrera estructural que dificulta la modernización del sector empresarial y limita sus posibilidades de crecimiento. Asimismo, la complejidad percibida al intentar adoptar nuevas tecnologías de la información termina siendo un obstáculo que desanima a muchas empresas [3]. Las interfaces técnicas de las herramientas de analítica requieren conocimientos especializados en bases de datos, estadística y ciencia de datos que frecuentemente no están al alcance del perfil medio de los trabajadores de una PyME, traduciéndose en una subutilización de la información organizacional.

Incluso cuando las organizaciones logran implementar soluciones analíticas, la falta de una cultura basada en datos y la escasez de competencias internas generan un retorno de inversión insatisfactorio [4]. En consecuencia, existe una necesidad urgente de soluciones de bajo código o lenguaje natural que permitan democratizar el acceso a la información estructurada.

Frente a este panorama, la Inteligencia Artificial Generativa emerge como una tecnología catalizadora para resolver esta problemática. La capacidad de los grandes modelos de lenguaje (LLMs) para procesar información y generar código facilita la creación de sistemas analíticos más accesibles [5]. Proyecciones recientes indican que la automatización mediante agentes de IA y la inteligencia de decisión son tendencias clave para reducir la brecha analítica en el sector corporativo [6].

Este trabajo presenta BIGENIA, una arquitectura

de Inteligencia de Negocios diseñada para mitigar estos obstáculos mediante el uso de lenguaje natural. El sistema implementa un agente de *Generative Business Intelligence* (GenBI) capaz de interpretar consultas expresadas en lenguaje cotidiano, convertirlas en código estructurado y desplegar los resultados mediante gráficos interactivos. Para validar el funcionamiento de esta arquitectura y su precisión en la extracción de datos, se utiliza un conjunto de datos relacional. De este modo, se busca proporcionar una interfaz que elimine la necesidad de intermediarios técnicos o conocimientos avanzados en SQL.

El presente artículo se estructura de la siguiente manera: la Sección 2 presenta la justificación del trabajo. La Sección 3 detalla el marco teórico fundamental. La Sección 4 revisa el estado del arte en IA aplicada a bases de datos. Posteriormente, la Sección 5 describe de forma exhaustiva la arquitectura de la aplicación, desglosando los agentes y componentes integrados. Finalmente, se exponen las conclusiones obtenidas y se proponen futuras líneas de investigación.

2. Justificación

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed at tortor maximus, convallis lectus at, fringilla nibh.

3. Marco Teórico

3.1. Arquitectura de datos moderna y capas semánticas

La implementación de sistemas de Inteligencia de Negocios Generativa (GenBI) necesita separar la información de las operaciones diarias de aquella usada para el análisis. Las bases de datos transaccionales, diseñadas para el registro rápido y seguro de datos crudos, no contienen la lógica del negocio. Conectar modelos de lenguaje directamente a estas bases de datos aumenta el riesgo de generar métricas incorrectas o respuestas inventadas, ya que la Inteligencia Artificial desconoce las reglas propias de la organización [7]. Investigaciones recientes señalan que la complejidad de las tablas, los nombres poco claros de las columnas y la falta de contexto son las principales causas de error en los sistemas actuales que traducen texto a SQL [8].

Para solucionar este problema, la arquitectura de datos moderna utiliza almacenes analíticos y construye una capa semántica, que funciona como un puente de comunicación. Esta capa se encarga de traducir la complejidad técnica de las tablas y sus relaciones en conceptos de negocio claros y estandarizados. Estudios actuales sugieren automatizar la creación de esta capa utilizando sistemas multi-agente. En este esquema, los modelos de lenguaje organizan la información relacional en estructuras mucho más fáciles de comprender,

logrando una alta precisión al integrar los datos de la empresa [9].

Dentro del análisis de datos, la capa semántica proporciona a la Inteligencia Artificial un entorno de trabajo seguro y controlado. En lugar de requerir que el modelo deduzca cómo relacionar múltiples tablas en bases de datos complejas, el agente interactúa únicamente con información validada y simplificada [8]. Recientemente, este enfoque se ha integrado en sistemas que emplean modelos de lenguaje para procesar consultas. En estos casos, la capa semántica permite interpretar con mayor claridad la intención del usuario antes de generar el código. Esto asegura la obtención de resultados confiables y estrictamente alineados con las reglas de la organización [10].

3.2. Conversión de texto a lenguaje SQL

La conversión de texto a lenguaje estructurado (Text-to-SQL) es una herramienta clave para facilitar el acceso a la información en las empresas. Permite que usuarios sin conocimientos de programación puedan consultar bases de datos relacionales utilizando lenguaje natural. Este proceso analiza la estructura y el significado de las preguntas para entender qué datos se están solicitando [11]. Sin embargo, el principal desafío técnico es resolver la ambigüedad de las palabras y entender exactamente qué quiere el usuario, especialmente cuando las bases de datos tienen nombres o estructuras poco descriptivas [12].

Para mejorar su funcionamiento, los sistemas actuales utilizan técnicas de vinculación de esquemas (*Schema Linking*). Este método identifica las entidades mencionadas en la pregunta del usuario y las asocia correctamente con las tablas y columnas de la base de datos, reduciendo las confusiones [13]. Estudios recientes sugieren el uso de redes y análisis de atención para mejorar esta conexión. Esto ayuda a que el modelo comprenda cómo se relacionan los datos entre sí antes de empezar a escribir el código SQL [14].

Cuando las consultas se vuelven más complejas, los sistemas modernos recurren a estrategias de razonamiento paso a paso. En lugar de generar todo el código de una sola vez, los modelos dividen la pregunta en problemas más pequeños y validan cada paso antes de dar la respuesta final [15]. Esta estrategia, sumada al uso de modelos entrenados específicamente para programar, aumenta notablemente la precisión de los resultados en entornos corporativos reales que manejan grandes volúmenes de datos [16].

3.3. Generación Aumentada por Recuperación (RAG)

Para superar los límites de memoria de los modelos de lenguaje y evitar que inventen información, se utiliza una arquitectura llamada Generación Aumentada por Recuperación (RAG). Esta técnica permite

que la Inteligencia Artificial consulte documentos externos antes de generar una respuesta, reduciendo su dependencia exclusiva de los datos con los que fue entrenada originalmente. En entornos corporativos, esto es fundamental para garantizar que el modelo responda basándose únicamente en la información validada de la propia empresa [17].

El funcionamiento interno de RAG se divide en dos fases: recuperación y generación. Primero, la información de la empresa se transforma en representaciones matemáticas llamadas vectores y se guarda en una base de datos especial. Cuando un usuario hace una pregunta, el sistema busca matemáticamente cuáles son los fragmentos de información más similares. Luego, esta información recuperada se le entrega al modelo de lenguaje para que tenga el contexto exacto antes de formular su respuesta [18].

En el análisis de datos, RAG se emplea para buscar dinámicamente descripciones de tablas o ejemplos de consultas que ya han sido resueltas por expertos. Al incluir estos ejemplos exitosos junto con la pregunta del usuario, el modelo mejora significativamente su capacidad para entender el problema y redactar el código correcto. Este enfoque disminuye los errores y facilita la resolución automática de preguntas analíticas con un alto grado de precisión [19].

3.4. Sistemas Multi-Agente

El avance de la Inteligencia Artificial ha permitido pasar de modelos que simplemente responden preguntas a agentes autónomos. Un agente se define por su capacidad de mantener un objetivo, recordar conversaciones pasadas, planificar tareas paso a paso y utilizar herramientas externas, como ejecutar código o consultar bases de datos. Esta capacidad de razonar y actuar de forma independiente permite resolver problemas mucho más complejos que la simple generación de texto [20].

En lugar de depender de un solo agente para hacer todo el trabajo, las arquitecturas modernas utilizan sistemas multi-agente. En este esquema, el problema se divide entre varios especialistas virtuales que colaboran entre sí. Por ejemplo, un agente puede encargarse exclusivamente de escribir el código SQL, mientras que otro actúa como revisor para detectar errores sintácticos antes de ejecutar la consulta. Esta división del trabajo imita el flujo de desarrollo de un equipo humano y aumenta notablemente la calidad del resultado final [21].

En el contexto de la Inteligencia de Negocios, la orquestación de múltiples agentes representa un pilar para la automatización segura. Al distribuir la responsabilidad, los sistemas pueden enfrentar de manera colaborativa los desafíos de interactuar con bases de datos empresariales. Esta supervisión cruzada reduce los errores del modelo y asegura que las decisiones analíticas se basen en datos precisos y correctamente procesados [6].

3.5. Generación automática de visualizaciones

En la Inteligencia de Negocios, la representación gráfica de los datos es el paso final indispensable tras realizar una consulta. La investigación actual busca que los modelos de lenguaje no solo devuelvan tablas con números, sino que traduzcan las peticiones del usuario directamente en visualizaciones de datos. El objetivo es que personas sin conocimientos técnicos puedan crear y consumir gráficos interactivos simplemente describiendo lo que necesitan ver [22].

Para lograr que la Inteligencia Artificial diseñe gráficos precisos, se utilizan metodologías que obligan al modelo a razonar su proceso creativo. Antes de escribir el código de visualización, el sistema planifica lógicamente qué tipo de gráfico es el más adecuado para los datos obtenidos, qué variables irán en cada eje y qué paleta de colores utilizar. Este razonamiento paso a paso mejora la relación entre la pregunta original y el diseño final [15].

A pesar de estos avances, los sistemas actuales suelen utilizar herramientas de código ligeras para generar los tableros, lo que facilita el desarrollo pero introduce nuevas limitaciones [23]. Problemas como escalas incorrectas en los ejes o la falta de opciones avanzadas de interactividad son comunes. Por lo tanto, mejorar la fidelidad gráfica y lograr que estos tableros compitan con las herramientas tradicionales sigue siendo un área que requiere mayor investigación [22].

3.6. Evaluación y modelos como jueces (LLM-as-a-Judge)

Evaluar la calidad de un sistema de Inteligencia de Negocios Generativa (GenBI) es una tarea sumamente compleja, ya que el proceso abarca desde la extracción de datos hasta el diseño visual. Tradicionalmente, la evaluación se limitaba a la etapa de generación de consultas (Text-to-SQL), midiendo si el código escrito por el modelo era idéntico a una respuesta esperada. Como esta métrica resultó demasiado rígida, la industria migró hacia medir la precisión de ejecución (*Execution Accuracy*), verificando únicamente si los datos devueltos son correctos [16]. No obstante, cuando el sistema también genera tableros y gráficos, estas métricas matemáticas se vuelven insuficientes.

Dado que es imposible revisar manualmente miles de resultados que combinan código SQL, estructuras de datos y representaciones visuales, ha cobrado fuerza la estrategia de utilizar modelos de lenguaje como jueces automatizados (*LLM-as-a-Judge*). En este enfoque, se emplea un modelo avanzado para leer la pregunta original del usuario y evaluar semánticamente si todo el flujo tiene sentido. El modelo actúa como un auditor humano, verificando que el razonamiento detrás de la respuesta sea lógicamente válido y útil para el negocio [24].

Esta metodología de evaluación resulta indispensable para auditar ciclos analíticos de principio a fin. Al utilizar un modelo como juez, el sistema puede calificar de manera integral tanto la exactitud de los datos extraídos como la coherencia de la explicación textual y la claridad del gráfico generado. Contar con este nivel de evaluación flexible permite realizar análisis comparativos sólidos entre diferentes arquitecturas, garantizando que el tablero final cumpla con los estándares de calidad corporativos [24].

4. Estado del Arte

4.1. Avances y limitaciones recientes en Text-to-SQL

La literatura reciente destaca una evolución significativa en la tarea de Text-to-SQL, impulsada por nuevos marcos de trabajo que mejoran la alineación entre la intención del usuario y el esquema de la base de datos [13]. Estos avances se apoyan en la capacidad de los grandes modelos de lenguaje para procesar semántica compleja, superando ampliamente a los enfoques tradicionales basados en reglas heurísticas o análisis sintáctico superficial [11].

En términos metodológicos, arquitecturas recientes como Gen-SQL proponen mecanismos eficientes para acortar la brecha entre el lenguaje natural y la base de datos, optimizando la identificación de tablas y columnas relevantes previo a la generación del código [13]. De manera complementaria, marcos de trabajo industriales integran procesos estandarizados que buscan garantizar la consistencia sintáctica y semántica de las consultas generadas, un factor crítico para su adopción en ecosistemas de datos productivos [14].

A pesar de este progreso empírico, la resolución de consultas analíticas que involucren múltiples cruces (*joins*), subconsultas anidadas y operaciones matemáticas sigue siendo un problema abierto [11]. La introducción de entornos de evaluación rigurosos, como el *benchmark* Spider 2.0, expone estas vulnerabilidades al someter a los modelos a tareas realistas y altamente desafiantes. Los resultados de estas pruebas demuestran que la precisión de ejecución (*Execution Accuracy*) cae drásticamente en esquemas masivos, lo que subraya la necesidad de integrar mecanismos de validación externa, como la orquestación de agentes, para asegurar la fiabilidad operativa [16].

4.2. Despliegue de RAG y agentes en entornos corporativos

El paso de entornos controlados de laboratorio a implementaciones productivas representa uno de los focos principales de la investigación actual en Inteligencia de Negocios. Para que los modelos generativos sean viables corporativamente, la literatura subraya la necesidad de

acoplar técnicas de recuperación de contexto que limiten el conocimiento a dominios específicos y mitiguen las alucinaciones [25].

Estudios empíricos recientes documentan la efectividad de la Generación Aumentada por Recuperación (RAG) en escenarios de alta criticidad. Por ejemplo, la implementación de estas arquitecturas en instituciones financieras ha demostrado mejoras sustanciales en la gestión eficiente de *insights*, permitiendo consultas seguras sobre repositorios documentales y esquemas de bases de datos sin exponer la privacidad de la información [17].

Sin embargo, el despliegue de sistemas orquestados por agentes de IA enfrenta obstáculos operativos significativos, especialmente en PyMEs y ecosistemas de la Industria 4.0 [25]. Las investigaciones advierten que las barreras de infraestructura, la falta de gobernanza de datos y la complejidad de gestionar múltiples agentes autónomos dificultan la estabilidad del sistema [20]. Esto evidencia que el desafío actual no radica únicamente en la generación de la respuesta, sino en la arquitectura de despliegue que sostiene al agente interactuando con el entorno empresarial.

4.3. Fronteras en la visualización automatizada de datos

En el dominio de la representación gráfica, los enfoques más vanguardistas aprovechan la capacidad de los grandes modelos de lenguaje para ir más allá de la consulta tabular y traducir comandos en lenguaje natural directamente hacia visualizaciones. El objetivo principal de estos esfuerzos es automatizar el ciclo final del Business Intelligence, permitiendo a usuarios no técnicos consumir información en formatos gráficos de manera interactiva [22].

Para lograr una traducción efectiva entre el lenguaje natural y las gramáticas de visualización, la investigación actual emplea metodologías de razonamiento escalonado (*Chain-of-Thought*). Frameworks como DeepVIS demuestran que obligar al modelo a planificar lógicamente el tipo de gráfico, los ejes y las agregaciones antes de generar el código reduce drásticamente la brecha semántica y mejora la precisión del diseño [15]. Paralelamente, surge una fuerte tendencia hacia la creación de arquitecturas ligeras, diseñadas específicamente para generar tableros completos sin la sobrecarga de herramientas de BI tradicionales [23].

No obstante, la literatura advierte que la utilidad real de estos sistemas aún está en fase de maduración. Las alucinaciones en la escala de los ejes, la selección incorrecta de paletas de colores y la falta de interactividad avanzada son problemas recurrentes. Además, la fidelidad gráfica comparativa sigue siendo un obstáculo crítico, limitando la confianza del usuario final en las interfaces generadas íntegramente por Inteligencia Artificial [22].

4.4. Desafíos abiertos en la evaluación comparativa

A pesar de los rápidos avances en la generación de código y visualizaciones, la evaluación de sistemas de Inteligencia de Negocios Generativa (GenBI) constituye uno de los mayores cuellos de botella metodológicos. Las aproximaciones clásicas, basadas en métricas deterministas, resultan insuficientes para capturar los matices del razonamiento analítico, ya que una respuesta válida en BI puede presentarse con múltiples variaciones estructurales y visuales [22].

Para abordar este problema, investigaciones publicadas entre 2024 y 2026 enfatizan la necesidad de ejecutar análisis comparativos exhaustivos sobre el rendimiento de distintos modelos fundacionales, evaluando su capacidad para operar dentro de flujos de trabajo analíticos reales [24]. En este contexto, el uso de LLMs como jueces automatizados se postula como la técnica más prometedora para evaluar semánticamente tanto el código SQL generado como la calidad y coherencia de las representaciones gráficas.

A pesar de estos esfuerzos, la carencia de marcos de evaluación integrales que auditen el ciclo de vida completo de los datos —desde la recuperación del esquema y la orquestación multi-agente, hasta el diseño final del tablero— representa una brecha crítica en el estado del arte. Esta limitación documentada subraya la relevancia de proponer nuevas arquitecturas supervisadas que garanticen la trazabilidad y la corrección técnica en entornos analíticos generativos.

5. Desarrollo

5.1. Arquitectura de la aplicación

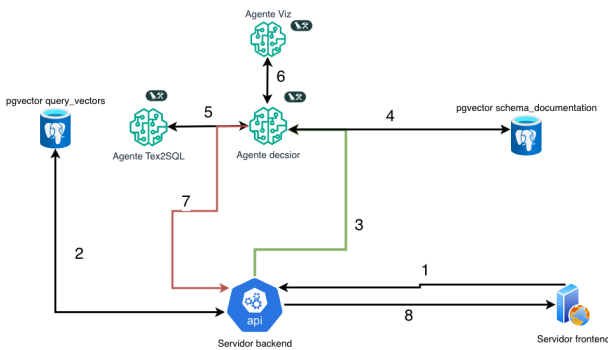


Figura 1: Diagrama de arquitectura del sistema BIGENIA. Se detallan los 8 pasos de comunicación entre los agentes, el servidor y las bases de datos vectoriales.

El sistema BIGENIA ha sido diseñado bajo un modelo de flujo de trabajo basado en agentes (*Agentic Workflow*), estructurado en diferentes componentes especializados. Como se ilustra en la Figura 1, la comu-

nicación entre las capas y los agentes sigue un proceso ordenado de 8 pasos:

1. **Recepción de la consulta:** El servidor frontend (interfaz web) captura la pregunta del usuario en lenguaje natural y envía la petición HTTP al servidor backend (API REST).
2. **Caché Semántico (*query_vectors*):** El servidor backend consulta la base de datos vectorial (*pgvector query_vectors*) para comprobar si existe una pregunta semánticamente idéntica que ya haya sido procesada. Si ocurre un *cache hit*, se reutiliza la consulta SQL asociada, ahorrando tiempo de procesamiento.
3. **Delegación al Orquestador:** Si la consulta es nueva (*cache miss*), la API transfiere la responsabilidad lógica al **Agente Decisor**, el componente central encargado de dirigir el flujo de trabajo.
4. **Extracción de Contexto (*schema_documentation*):** El Agente Decisor consulta la segunda base de datos vectorial (*pgvector schema_documentation*) para obtener los metadatos relevantes del esquema relacional (DDLs, documentación de tablas y reglas de negocio) que servirán de contexto.
5. **Interacción con Text2SQL:** El Agente Decisor envía el contexto recuperado y la pregunta original al **Agente Text2SQL** (potenciado por Vanna AI) para que el Modelo de Lenguaje Grande formule la sentencia SQL correcta.
6. **Decisión de Visualización (Agente Viz):** En paralelo o subsecuentemente, el Agente Decisor provee la estructura de los datos obtenidos al **Agente Viz**, quien decide la forma óptima de representarlos visualmente y genera el código JSON de configuración para Plotly.
7. **Retorno de Resultados:** Una vez procesado, se transfiere la respuesta completa (incluyendo la consulta SQL, los datos y la configuración del gráfico generado) de vuelta al servidor backend, preparándola para ser enviada al cliente.
8. **Respuesta Final:** El servidor backend envía este resultado consolidado al servidor frontend, donde la interfaz finalmente renderiza el gráfico interactivo y presenta la información al usuario.

Esta arquitectura desacoplada permite que cada agente y componente de la base de datos asuma un rol altamente especializado, facilitando el mantenimiento y permitiendo futuras actualizaciones tecnológicas (como el reemplazo del modelo LLM) sin comprometer la integridad del flujo de datos completo.

Adicionalmente, se ha integrado **LangSmith** en el flujo de trabajo como plataforma de observabilidad para

los modelos de lenguaje subyacentes. Esta herramienta resulta fundamental para monitorear, rastrear y depurar las ejecuciones de los agentes en tiempo real. Gracias a LangSmith, es posible analizar el rendimiento de los *prompts*, medir los tiempos de latencia, auditar el consumo de tokens y detectar anomalías o alucinaciones durante la generación de sentencias SQL y gráficos, garantizando así la calidad y confiabilidad continua de las respuestas de BIGENIA.

5.2. Agente Decisor

El Agente Decisor constituye el punto de entrada y orquestador central del pipeline de BIGENIA. Su función principal es recibir la consulta del usuario en lenguaje natural, junto con el historial de conversación de la sesión activa, y determinar la acción apropiada a ejecutar. Este componente actúa como director del flujo de trabajo, delegando responsabilidades a los agentes especializados según corresponda.

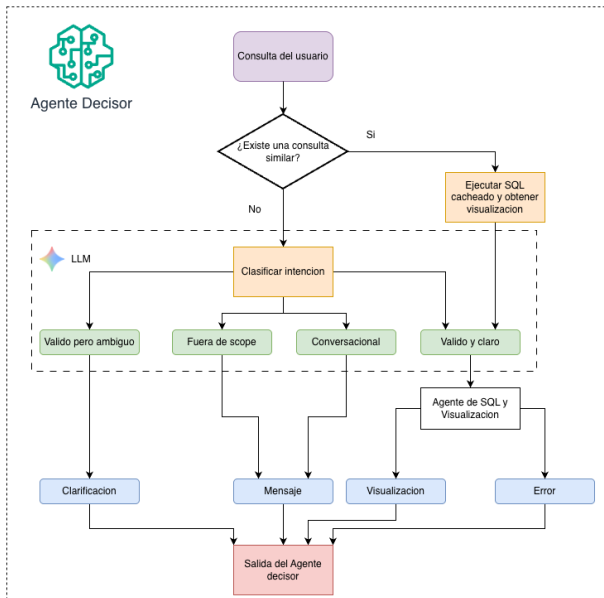


Figura 2: Diagrama de flujo del Agente Decisor. Según la clasificación de intención, el agente enruta la consulta hacia el pipeline de datos, solicita clarificación o emite un mensaje directo.

5.2.1. Clasificación de intención

El mecanismo central del Agente Decisor es un clasificador de intención que utiliza Gemini con salida estructurada (*structured output*). Esto garantiza que la respuesta del modelo se ajuste a un esquema predefinido, eliminando la necesidad de parseo manual de texto libre y reduciendo errores de interpretación.

El clasificador analiza la consulta del usuario junto con el contexto conversacional y la asigna a exactamente una de cuatro categorías posibles:

- **Consulta válida y clara:** La solicitud es suficientemente específica como para ser traducida a una sentencia SQL sin ambigüedades. El agente continúa con el pipeline completo: generación de SQL, ejecución sobre la base de datos y generación de la visualización correspondiente.
- **Consulta válida pero ambigua:** La solicitud está relacionada con el dominio de datos, pero carece de la especificidad necesaria para construir una consulta precisa. El agente genera una pregunta de clarificación y la devuelve al usuario. Para evitar ciclos infinitos de clarificación, el sistema implementa una guardia que permite un máximo de una pregunta de clarificación consecutiva; si el usuario ya recibió una clarificación en el turno anterior, el agente asume la interpretación más probable y avanza con el pipeline.
- **Fuera de alcance:** La consulta no guarda relación con el dominio de datos disponible, o solicita operaciones destructivas sobre la base de datos. El agente responde con un mensaje informativo indicando que la solicitud excede las capacidades del sistema.
- **Conversacional:** La entrada corresponde a saludos, despedidas, agradecimientos u otros comentarios de cortesía que no requieren procesamiento analítico. El agente responde con un mensaje apropiado.

5.2.2. Resolución contextual de consultas

Cuando la consulta del usuario es clasificada como válida pero depende del historial de conversación para ser interpretada correctamente (por ejemplo, “lo mismo pero agrupado por mes” o “ahora filtralo por Argentina”), el clasificador genera una consulta auto-contenida que reemplaza a la original. Esta consulta reformulada incorpora toda la información necesaria del contexto previo, permitiendo que las etapas posteriores del pipeline operen de forma independiente, sin necesidad de acceder al historial conversacional.

5.2.3. Tipos de respuesta

El Agente Decisor produce uno de cuatro tipos de respuesta, dependiendo de la categoría de intención detectada y del resultado del procesamiento:

- **Visualización:** Incluye la sentencia SQL generada, los datos resultantes y la configuración del gráfico interactivo. Se emite cuando el pipeline completo se ejecuta exitosamente.
- **Clarificación:** Contiene una pregunta dirigida al usuario para refinar su solicitud. Se emite para consultas ambiguas.

- **Mensaje:** Respuesta textual informativa. Se emite para consultas fuera de alcance, conversacionales, o cuando la consulta SQL se ejecutó correctamente pero no arrojó resultados.
- **Error:** Información sobre una falla en el pipeline. Se emite cuando alguna etapa del procesamiento falla de manera irrecuperable.

Adicionalmente, cuando el caché semántico del sistema detecta una consulta previamente procesada (*cache hit*), el Agente Decisor omite las etapas de clasificación y generación de SQL, ejecutando directamente la sentencia almacenada. Esto reduce significativamente el tiempo de respuesta y el consumo de recursos del modelo de lenguaje.

5.3. Agente de Text2SQL

El Agente de Text2SQL es el componente responsable de la traducción de consultas en lenguaje natural a sentencias SQL ejecutables, comúnmente denominado *Text-to-SQL*. Este agente utiliza Vanna AI como framework de generación y Gemini como modelo de lenguaje, operando sobre una base de datos PostgreSQL.

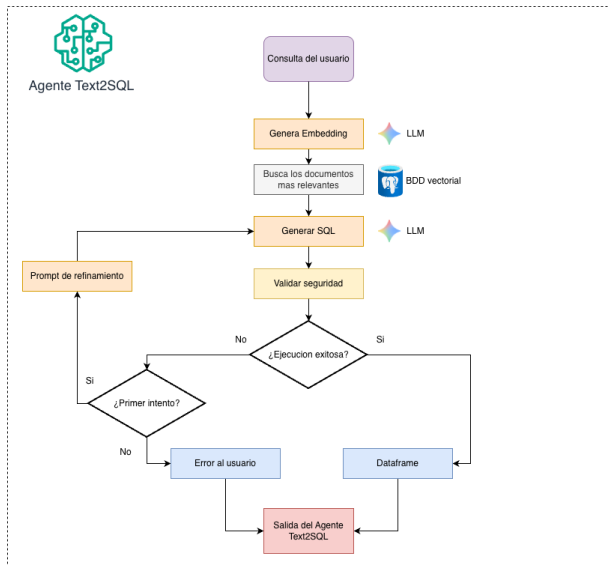


Figura 3: Ciclo de generación y reintento del Agente de Text2SQL. La sentencia SQL generada se valida y ejecuta; en caso de fallo, se construye un prompt de refinamiento para una segunda invocación del modelo.

5.3.1. Inyección de contexto semántico

Antes de invocar al modelo de lenguaje para la generación de SQL, el sistema emplea un patrón de *Retrieval-Augmented Generation* (RAG) para enriquecer el prompt con información contextual relevante del esquema relacional.

El proceso consiste en generar un *embedding* vectorial de la consulta del usuario mediante un modelo de embeddings dedicado y buscar, por distancia de coseno en la base de datos vectorial *schema_documentation*, los documentos de esquema semánticamente más relevantes. Estos documentos contienen descripciones enriquecidas de tablas y columnas que fueron previamente generadas de forma automática mediante un proceso de documentación semántica del esquema.

Los documentos recuperados se inyectan directamente en el prompt del modelo, proporcionando al LLM un contexto preciso sobre la estructura y semántica de los datos disponibles. Esta estrategia reduce significativamente las alucinaciones del modelo, mejorando la selección de tablas y columnas apropiadas y la construcción de sentencias SQL sintácticamente correctas.

5.3.2. Generación de SQL

El modelo de lenguaje recibe un prompt cuidadosamente diseñado que combina la consulta del usuario con el contexto semántico recuperado. Este prompt incluye reglas estrictas de generación:

- Solo se permiten sentencias **SELECT** (operaciones de solo lectura).
- Se especifica el esquema y las convenciones de nomenclatura de la base de datos.
- Los identificadores de tablas y columnas deben respetar el formato esperado por PostgreSQL.
- La temperatura del modelo se configura en 0.0 para maximizar el determinismo de las respuestas.

5.3.3. Validación de seguridad

Toda sentencia SQL generada por el modelo es sometida a un proceso de validación de seguridad antes de su ejecución. Este validador realiza un análisis sintáctico de la sentencia mediante la biblioteca *sqlparse* y verifica que se trate exclusivamente de una operación **SELECT**. Adicionalmente, se realiza un escaneo lexicográfico para detectar y bloquear palabras clave asociadas a operaciones destructivas (**DELETE**, **DROP**, **UPDATE**, **INSERT**, **ALTER**, entre otras). Si se detecta una sentencia prohibida, se interrumpe la ejecución inmediatamente, garantizando la integridad de los datos.

5.3.4. Mecanismo de reintento

Un aspecto clave del diseño del sistema es su capacidad de autocorrección ante fallos de ejecución SQL. Cuando una sentencia generada por el modelo falla al ejecutarse sobre la base de datos (por ejemplo, debido a un nombre de columna incorrecto o una sintaxis incompatible), el sistema implementa un mecanismo de reintento automático:

1. Se captura el mensaje de error devuelto por PostgreSQL.
2. Se construye un prompt de refinamiento que incluye tres elementos: la consulta original en lenguaje natural, la sentencia SQL que falló y el mensaje de error de la base de datos.
3. Se realiza una segunda invocación al modelo de lenguaje con este contexto de error, solicitando una versión corregida de la sentencia SQL.
4. La nueva sentencia se somete nuevamente al proceso de validación de seguridad y ejecución.

El sistema admite un máximo de dos intentos (el original y uno de corrección) gestionados de manera transparente para el usuario. Si ambos fallan, se reporta el error final mediante el Agente Decisor.

5.3.5. Manejo de resultados vacíos

Si la ejecución SQL es exitosa pero no devuelve registros, el Agente Decisor informa directamente la ausencia de datos sin invocar al Agente de Visualización, optimizando recursos.

5.4. Agente de visualización

El Agente de Visualización transforma los resultados SQL y la consulta original en un gráfico interactivo en formato JSON compatible con Plotly.

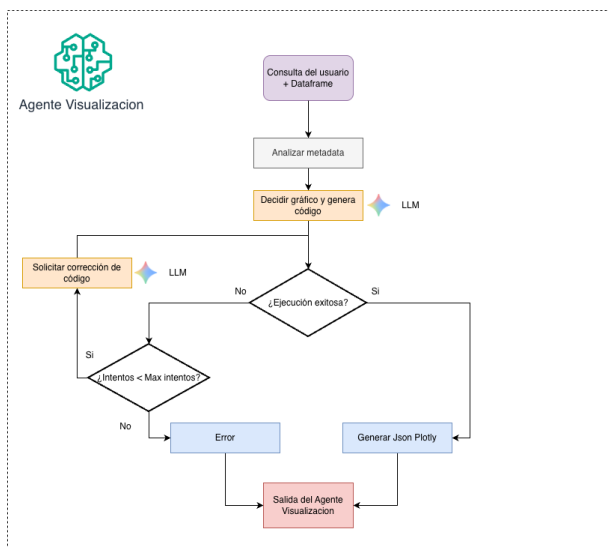


Figura 4: Pipeline de generación del Agente de Visualización. El código Plotly generado se ejecuta y valida; si falla, se solicita una corrección automática al modelo, repitiendo el ciclo hasta agotar los intentos.

5.4.1. Pipeline de generación

El proceso de generación de una visualización sigue un pipeline de cinco etapas secuenciales:

1. **Validación de datos:** Se verifica que el conjunto de datos recibido no esté vacío ni presente anomalías estructurales que impidan su representación gráfica.
2. **Análisis del conjunto de datos:** Se extrae metadata detallada del DataFrame, incluyendo las dimensiones, los tipos de datos de cada columna (numérica, categórica, temporal), la cantidad de valores únicos, valores de muestra y la presencia de valores nulos. Esta metadata permite al modelo de lenguaje tomar decisiones informadas sobre la representación visual más adecuada.
3. **Decisión y generación de código:** La metadata extraída junto con la solicitud del usuario se envían al modelo de lenguaje (Gemini) utilizando salida estructurada. El modelo selecciona el tipo de gráfico más apropiado entre las opciones disponibles (barras, líneas, torta, dispersión, histograma, mapa de calor y diagrama de caja) y genera código Python ejecutable utilizando la biblioteca Plotly. La respuesta estructurada incluye además un razonamiento que justifica la elección del tipo de gráfico.
4. **Ejecución con loop de corrección:** El código generado se ejecuta en un entorno controlado para producir la figura. Si la ejecución falla (por errores de sintaxis, errores en tiempo de ejecución o producción de un gráfico vacío), el sistema envía automáticamente el código fallido junto con el mensaje de error al modelo de lenguaje, solicitando una versión corregida. Este ciclo de corrección se repite hasta un número máximo de intentos configurable, permitiendo al sistema recuperarse de errores menores en la generación de código.
5. **Serialización:** Una vez obtenida una figura válida, se serializa a formato JSON de Plotly. Esta representación es directamente interpretable por la biblioteca Plotly.js en el frontend, permitiendo el renderizado de gráficos interactivos con capacidades de zoom, filtrado y exploración de datos.

5.4.2. Razonamiento de la decisión

Cada visualización generada incluye una explicación en lenguaje natural que detalla por qué se seleccionó un tipo de gráfico particular dada la naturaleza de los datos y la solicitud del usuario. Este razonamiento se presenta al usuario junto con la visualización, proporcionando transparencia sobre el proceso de decisión del sistema y facilitando la comprensión de la representación elegida.

5.4.3. Edición generativa de gráficos

Una capacidad distintiva del agente es la posibilidad de modificar visualizaciones existentes mediante instrucciones en lenguaje natural. El usuario puede solicitar cambios específicos sobre un gráfico previamente generado, tales como la modificación de colores, la aplicación de filtros, el cambio de tipo de gráfico o el ajuste de títulos y etiquetas. El agente recibe el código fuente del gráfico original junto con la nueva instrucción y el historial de conversación, generando una versión modificada que preserva el contexto del gráfico base. El código resultante también se somete al loop de corrección antes de ser devuelto al usuario.

5.4.4. Salida del agente

El resultado del Agente de Visualización incluye: el gráfico serializado en formato JSON (listo para renderizar en el frontend), el código fuente Plotly que lo genera, el tipo de gráfico seleccionado, el razonamiento de la decisión y métricas de ejecución como la cantidad de intentos realizados y el tiempo de procesamiento total. Esta información completa es devuelta al Agente Decisor, quien la consolida en la respuesta final enviada al usuario.

5.5. Interfaz de usuario

La interfaz de usuario fue diseñada priorizando la simplicidad y la capacidad de respuesta inmediata. Se utilizó **React** como biblioteca principal de frontend, complementada con **Tailwind CSS** para el estilizado y **Zustand** para la gestión de estado global.

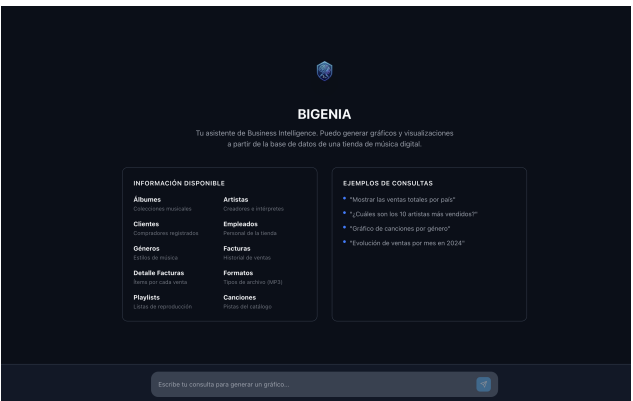


Figura 5: Pantalla de inicio de la aplicación con guía de uso.

5.5.1. Decisiones de Diseño y UX/UI

- **Estética Moderna y Minimalista:** Se implementó un diseño con bordes redondeados, utilizando una paleta de colores coherente que soporta tanto el modo claro como el modo oscuro (ver Figura 6).

Esta dualidad no es solo estética, sino que mejora la accesibilidad y reduce la fatiga visual según la preferencia del usuario.

- **Retroalimentación Inmediata:** Dado que la generación de gráficos mediante LLMs puede tomar varios segundos, se integraron indicadores de carga (*spinners* y *skeleton screens*) para mantener al usuario informado sobre el proceso en curso.
- **Interactividad de los Datos:** En lugar de imágenes estáticas, los gráficos se renderizan utilizando **Plotly.js**. Esto permite que el usuario pueda hacer zoom, filtrar series de datos y ver detalles exactos al pasar el cursor sobre los elementos.

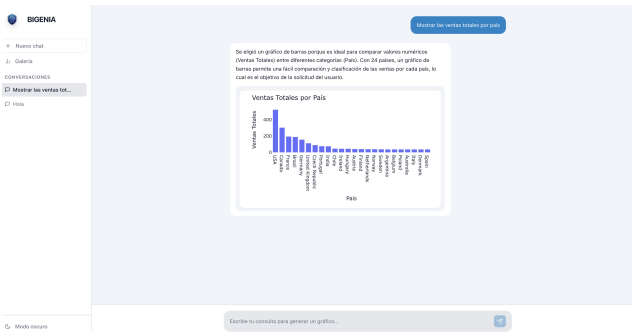


Figura 6: Visualización interactiva generada en modo claro.

5.5.2. Persistencia y Organización

Para facilitar el flujo de trabajo del analista, la aplicación cuenta con un historial de conversaciones y una **Galería de Gráficos** (Figura 7). Esta galería permite acceder rápidamente a las visualizaciones más importantes sin tener que buscar en el historial del chat, separando la lógica de conversación de los activos visuales generados.

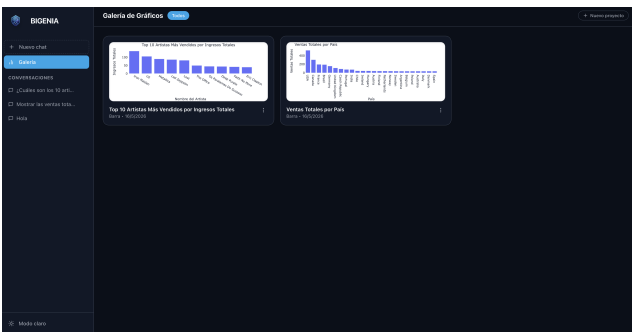


Figura 7: Galería centralizada para la gestión de activos visuales.

5.5.3. Edición Generativa

Una característica distintiva de la interfaz es la capacidad de modificar gráficos existentes mediante lenguaje natural. El usuario puede citar un gráfico previo y solicitar cambios específicos (ej. colores, filtros o tipos de gráfico), lo que se traduce en una experiencia de edición fluida e intuitiva (Figura 8).



Figura 8: Flujo de edición generativa sobre un gráfico previo.

6. Conclusiones

La investigación teórica y el desarrollo práctico de la arquitectura BIGENIA demuestran empíricamente la viabilidad de democratizar la analítica de datos en el sector PyME mediante el uso de Inteligencia Artificial Generativa. La implementación sobre un modelo de datos relacional de prueba confirma que la orquestación de agentes logra abstraer la complejidad técnica subyacente, consolidando una interfaz directa y efectiva entre el usuario de negocio y la base de datos.

A través de la metodología de evaluación *LLM-as-a-Judge*, aplicada sobre un conjunto estructurado de 50 casos de prueba, el sistema evidenció un cumplimiento robusto de los objetivos funcionales. El Agente de Base de Datos y el Agente Visualizador alcanzaron un desempeño sobresaliente en la resolución de consultas de baja y media complejidad, generando de forma autónoma tanto las sentencias SQL precisas como las representaciones gráficas adecuadas. Un hallazgo fundamental de las pruebas es la resiliencia del sistema: los mecanismos de control integrados lograron identificar y mitigar consistentemente las peticiones fuera de dominio (*guardrails*), garantizando un entorno de ejecución seguro y estrictamente acotado al contexto empresarial.

Si bien se identificó una degradación en la precisión de inferencia frente a consultas que exigen múltiples

uniones y subconsultas anidadas, este comportamiento permite establecer con claridad el límite operativo de la versión actual sin invalidar su utilidad estratégica. En definitiva, este trabajo corrobora que el reemplazo de interfaces técnicas tradicionales por modelos de procesamiento de lenguaje natural reduce drásticamente la curva de aprendizaje y dota de autonomía analítica real a organizaciones con recursos de TI limitados, sentando bases sólidas para el desarrollo de herramientas analíticas de próxima generación.

7. Líneas futuras de investigación

Agradecimientos

Referencias

- [1] Clarysabel Tovar. Business intelligence en las PyMEs: Obstáculos y enfoques de implementación. *Investigación sobre la aplicación de Business Intelligence*, 2011. URL https://www.palermo.edu/economicas/cbrs/pdf/pbr15/PBR_15_05_Tovar.pdf.
- [2] J. Cordero, J. Erazo, C. Narváez, and D. Cordeiro. Sistemas de información inteligente en MIPY-MES. *Revista de Sistemas de Información*, 2020. URL <https://dspace.ups.edu.ec/bitstream/123456789/26008/1/UPS-CT010867.pdf>.
- [3] A. S. Nurqamarani et al. Complejidad y falta de conocimiento en la adopción de TI. *Investigación Académica sin Frontera*, 2021. URL <https://revistainvestigacionacademicasinfrontera.unison.mx/index.php/RDIASF/article/download/554/721/2847>.
- [4] S. F. Wamba et al. Desafíos para la implementación efectiva de analítica de datos en mercados emergentes. *Multidisciplinary Collaborative Journal*, 2017. URL <https://mcjournal.editorialdosso.com/index.php/home/article/download/33/77>.
- [5] A. Barbon et al. Ia generativa y democratización de datos en grandes empresas y pymes. *International Journal for Multidisciplinary Research (IJFMR)*, 2024. URL <https://www.ijfmr.com/papers/2026/1/67384.pdf>.
- [6] Gartner. Predicciones de inteligencia de decisión y automatización por agentes de ia. *International Journal of Business Analytics*, 2025. URL https://www.researchgate.net/publication/400078248_A_Literature_Review_on_Data_Democratization_Self-Service_Business_Intelligence_and_Data_Literacy.

- [7] Solomon Negash. Fundamentos de inteligencia de negocios. *Communications of the Association for Information Systems*, 2004. URL <https://arxiv.org/html/2411.06102v3>.
- [8] Agapi Rissaki, Ilias Fountalis, Nikolaos Vasiloglou, and Wolfgang Gatterbauer. Towards agentic schema refinement. *arXiv preprint arXiv:2412.07786*, 2024. URL <https://arxiv.org/pdf/2412.07786>.
- [9] Ahmed Guerbas et al. A multi-agent system for semantic mapping of relational data to knowledge graphs. *arXiv preprint arXiv:2511.06455*, 2025. URL <https://arxiv.org/abs/2511.06455>.
- [10] S. Gunda et al. Making databases searchable with deep context. *arXiv preprint arXiv:2602.08320*, 2026. URL <https://arxiv.org/pdf/2602.08320>.
- [11] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and S. Nan. Natural language to sql: State of the art and open problems. In *Proceedings of the VLDB Endowment*, volume 18, 2024. URL <https://www.vldb.org/pvldb/vol18/p5466-luo.pdf>.
- [12] A. Srivastava and R. Gupta. Comparative analysis of llm-powered natural language interfaces in business intelligence. *International Journal of Computer Applications*, 186, 2026. URL <https://arxiv.org/pdf/2602.09912>.
- [13] B. Qin et al. Gen-sql: Efficient text-to-sql by bridging natural language question and database schema. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 3794–3807, 2024. URL <https://aclanthology.org/2025.coling-main.256.pdf>.
- [14] Tetiana Gladkykh and Kyrylo Kyrkov. Dattrics text2sql: A framework for natural language to sql query generation. Technical report, Dattrics Whitepaper v. 1.0, 2025. URL <https://arxiv.org/abs/2506.12234>.
- [15] Zhihao Shuai, Boyan Li, Siyu Yan, Yuyu Luo, and Weikai Yang. Deepvis: Bridging natural language and data visualization through chain-of-thought reasoning. *IEEE VIS 2025*, 2025. URL https://ieevis.org/year/2025/program/paper_f4073f80-c0bd-4d83-9c72-3dcfd060ef7f.html.
- [16] W. Lei et al. Spider 2.0 benchmark: Realistic and challenging text-to-sql tasks. *arXiv preprint arXiv:2411.07763*, 2024. URL <https://arxiv.org/pdf/2411.07763>.
- [17] David Alfonso Argüello Fiallos, Melanie Melissa Villalta Báez, Eduardo Cruz, et al. Rag aplicado a business intelligence para la gestión eficiente de insights dentro de una institución financiera ecuatoriana. 2025. URL <https://www.dspace.espol.edu.ec/bitstream/123456789/65807/1/T-115105%20POSTG107%20ARGUELLO-VILLALTA.pdf>.
- [18] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024. URL <https://arxiv.org/pdf/2312.10997>.
- [19] Linda Aluso and Onma Enyejo. Leveraging nlp and retrieval-augmented generation (rag) models for automated business intelligence query resolution. *International Journal of Scientific Research in Science, Engineering and Technology*, 11:534–557, 08 2024. doi: 10.32628/IJSRSET242439. URL https://www.researchgate.net/publication/401570857_Leveraging_NLP_and_Retrieval-Augmented_Generation_RAG_Models_for_Automated_Business_Intelligence_Query_Resolution.
- [20] R. Maddila et al. Agentes de ia para acceso a datos en business intelligence: Desafíos de despliegue. *International Journal for Multidisciplinary Research (IJFMR)*, 2024. URL <https://www.ijfmr.com/papers/2026/1/67384.pdf>.
- [21] Bing Wang et al. Mac-sql: A multi-agent collaborative framework for text-to-sql. *arXiv preprint arXiv:2312.11242*, 2024. URL <https://arxiv.org/pdf/2312.11242>.
- [22] Mario Perez-Montoro and Carlos Lopezosa. Ia generativa en visualización de datos: utilidad, desafíos y evaluación comparativa. *Revista Mediterránea de Comunicación*, 17(1):e30124–e30124, 2026. URL <https://rua.ua.es/server/api/core/bitstreams/1c74a055-e7cb-4aec-954e-cb3821d73290/content>.
- [23] Boshen Shi, Kexin Yang, Yuanbo Yang, Guang Chang, Ce Chi, Zhendong Wang, Xing Wang, and Junlan Feng. N12dashboard: A lightweight and controllable framework for generating dashboards with llms, 2026. URL <https://arxiv.org/pdf/2601.06126>.
- [24] Bhavesh Jaisinghani and Saurabh Aggarwal. Comparative performance analysis of large language models in generative business intelligence: Insights from llama3 and bamboollm. *International Journal For Multidisciplinary Research*, 6:November–December 2024, 12 2024. doi: 10.36948/ijfmr.2024.v06i06.33967. URL https://www.researchgate.net/publication/387736634_Comparative_Performance_

Analysis_of_Large_Language_Models_
in_Generative_Business_Intelligence_
Insights_from_Llama3_and_BambooLLM.

- [25] J. Bourdin. Modelos fundacionales y pymes: Oportunidades y desafíos de nlp en la industria 4.0. Technical report, Polytechnique Montréal, 2024. URL https://publications.polymtl.ca/59030/1/2024_Bourdin_NLP_SMEs_Industry_4.0_Opportunities.pdf.